Caching

```
import warnings
warnings.filterwarnings("ignore")
```

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

```
from pyspark.storagelevel import StorageLevel
from pyspark.sql.types import *
import pyspark.sql.functions as F
from pyspark.sql import SparkSession
```

```
spark = (
    SparkSession
    .builder
    .config("spark.driver.memory", "10g")
    .config("spark.sql.files.maxPartitionBytes", "268435456")
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    .master("local[*]")
    .getOrCreate()
)
sc = spark.sparkContext
sc.setLogLevel("ERROR")
```

```
customers_file = "../data/data_skew/customers.parquet"
df_customers = spark.read.parquet(customers_file)
```

```
df_customers.show(5, False)
```

```
+----------+-------------+---+------+----------+-----+-----------+
|cust_id   |name         |age|gender|birthday  |zip  |city       |
+----------+-------------+---+------+----------+-----+-----------+
|C007YEYTX9|Aaron Abbott |34 |Female|7/13/1991 |97823|boston     |
|C00B971T1J|Aaron Austin |37 |Female|12/16/2004|30332|chicago    |
|C00WRSJF1Q|Aaron Barnes |29 |Female|3/11/1977 |23451|denver     |
|C01AZWQMF3|Aaron Barrett|31 |Male  |7/9/1998  |46613|los_angeles|
|C01BKUFRHA|Aaron Becker |54 |Male  |11/24/1979|40284|san_diego  |
+----------+-------------+---+------+----------+-----+-----------+
only showing top 5 rows
```

```
df_base = (
    df_customers
    .filter(F.col("city") == "boston")
    .withColumn(
        "customer_group",
        F.when(
            F.col("age").between(20, 30),
            F.lit("young")
        )
        .when(
            F.col("age").between(31, 50),
            F.lit("mid")
        )
        .when(
            F.col("age") > 51,
            F.lit("old")
        )
        .otherwise(F.lit("kid"))
     )
    .select("cust_id", "name", "age", "gender", "birthday", "zip", "city", "customer_group")
)

df_base.cache()
df_base.show(5, False)
```

```
DataFrame[cust_id: string, name: string, age: string, gender: string, birthday: string, zip: string, city: string, customer_group:
string]
```

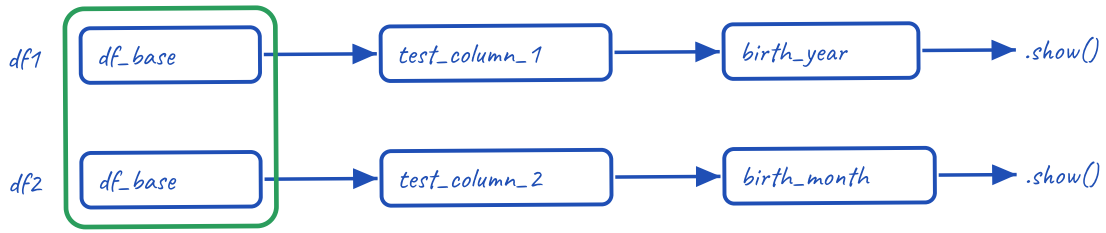
```
+----------+--------------+---+------+---------+-----+------+--------------+
|cust_id   |name          |age|gender|birthday |zip  |city  |customer_group|
+----------+--------------+---+------+---------+-----+------+--------------+
|C007YEYTX9|Aaron Abbott  |34 |Female|7/13/1991|97823|boston|mid           |
|C08XAQUY73|Aaron Lambert |54 |Female|11/5/1966|75218|boston|old           |
|C094P1VXF9|Aaron Lindsey |24 |Male  |9/21/1990|29399|boston|young         |
|C097SHE1EF|Aaron Lopez   |22 |Female|4/18/2001|82129|boston|young         |
|C0DTC6436T|Aaron Schwartz|52 |Female|7/9/1962 |57192|boston|old           |
+----------+--------------+---+------+---------+-----+------+--------------+
only showing top 5 rows
```

✍️ Definition: what `.cache()` does

- Marks this DataFrame: **"the next time anyone needs you, don't rebuild — read the stored copy".**
- It is **lazy**. The call above only wraps the plan in an **`InMemoryRelation`** node and returns the same DataFrame — that is the **`DataFrame[cust_id: string, ...]`** line printed above.
- The **`.show(5)`** under it is the **action** that actually fills the cache.
- What gets stored is **df_base after the filter and the CASE WHEN** — 517 Boston rows, not the 5,000 in the file.

⚠ **Gotcha:** **`.show(5)`** only materialises the partitions it had to touch. Use **`df.count()`** when you want the whole thing in the cache before the real work starts.

Lazy evaluation: what .show() really triggers



built TWICE without cache → cache it once

💡 Cache what you read **twice or more**. Caching a DataFrame used once is pure cost.

```
df1 = (
    df_base
    .withColumn("test_column_1", F.lit("test_column_1"))
    .withColumn("birth_year", F.split("birthday", "/").getItem(2))
)

df1.explain(True)
df1.show(5, False)
```

```
[parsed / analyzed / optimized plans trimmed - see spark/2_reading_query_plans.ipynb]

== Physical Plan ==
*(1) Project [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, customer_group#57, test_column_1 AS
test_column_1#307, split(birthday#47, /, -1)[2] AS birth_year#317 ④]
+- InMemoryTableScan ① [age#45, birthday#47, city#49, cust_id#43, customer_group#57, gender#46, name#44, zip#48]
      +- InMemoryRelation ② [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, customer_group#57],
StorageLevel(disk, memory, deserialized, 1 replicas) ③
            +- *(1) Project [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, CASE WHEN ((cast(age#45 as int) >= 20)
AND (cast(age#45 as int) <= 30)) THEN young WHEN ((cast(age#45 as int) >= 31) AND (cast(age#45 as int) <= 50)) THEN mid WHEN (cast(age#45
as int) > 51) THEN old ELSE kid END AS customer_group#57]
               +- *(1) Filter (isnotnull(city#49) AND (city#49 = boston))
                  +- *(1) ColumnarToRow
                     +- FileScan parquet [cust_id#43,name#44,age#45,gender#46,birthday#47,zip#48,city#49] Batched: true, DataFilters:
[isnotnull(city#49), (city#49 = boston)], Format: Parquet, Location: InMemoryFileIndex(1 paths)
[file:/Users/afaqueahmad/Documents/youtube/spark-experiments/data/data_..., PartitionFilters: [], PushedFilters: [IsNotNull(city),
EqualTo(city,boston)], ReadSchema: struct<cust_id:string,name:string,age:string,gender:string,birthday:string,zip:string,city:string>

+----------+--------------+---+------+---------+-----+------+--------------+-------------+----------+
|cust_id   |name          |age|gender|birthday |zip  |city  |customer_group|test_column_1|birth_year|
+----------+--------------+---+------+---------+-----+------+--------------+-------------+----------+
|C007YEYTX9|Aaron Abbott  |34 |Female|7/13/1991|97823|boston|mid           |test_column_1|1991      |
|C08XAQUY73|Aaron Lambert |54 |Female|11/5/1966|75218|boston|old           |test_column_1|1966      |
|C094P1VXF9|Aaron Lindsey |24 |Male  |9/21/1990|29399|boston|young         |test_column_1|1990      |
|C097SHE1EF|Aaron Lopez   |22 |Female|4/18/2001|82129|boston|young         |test_column_1|2001      |
|C0DTC6436T|Aaron Schwartz|52 |Female|7/9/1962 |57192|boston|old           |test_column_1|1962      |
+----------+--------------+---+------+---------+-----+------+--------------+-------------+----------+
only showing top 5 rows
```

✍️ Reading the physical plan after `cache()`

match the circles in the output above

- ① `InMemoryTableScan` ⟶ df1 reads the **cached block**, not the parquet file.
- ② `InMemoryRelation` ⟶ the cached block itself. Everything indented under it is the **recipe**, kept so Spark can rebuild a block that gets evicted or lost.
- ③ `StorageLevel(disk, memory, deserialized, 1 replicas)` ⟶ the default that `.cache()` picks = `MEMORY_AND_DISK_DESER`.
- ④ `split(birthday#47, /, -1)[2] AS birth_year#317` ⟶ the only **new** work df1 does on top of the cache.

⚠ Gotcha: `FileScan parquet` is still printed, under `InMemoryRelation`. That is the recipe, not a re-read. In the Spark UI the same node reports **0 rows, 0 ms, 0 bytes.**

💡 Plan text ≠ work done. The row counts in the SQL tab are the truth.

```python
df2 = (
    df_base
    .withColumn("test_column_2", F.lit("test_column_2"))
    .withColumn("birth_month", F.split("birthday", "/").getItem(1))
)

df2.explain(True)
df2.show(5, False)
```

```
[parsed / analyzed / optimized plans trimmed - see spark/2_reading_query_plans.ipynb]

== Physical Plan ==
*(1) Project [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, customer_group#57, test_column_2 AS
test_column_2#649, split(birthday#47, /, -1)[1] AS birth_month#659 ③]
+- InMemoryTableScan ① [age#45, birthday#47, city#49, cust_id#43, customer_group#57, gender#46, name#44, zip#48]
      +- InMemoryRelation [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, customer_group#57], StorageLevel(disk,
memory, deserialized, 1 replicas)
            +- *(1) Project [cust_id#43, name#44, age#45, gender#46, birthday#47, zip#48, city#49, CASE WHEN ② ((cast(age#45 as int)
>= 20) AND (cast(age#45 as int) <= 30)) THEN young WHEN ((cast(age#45 as int) >= 31) AND (cast(age#45 as int) <= 50)) THEN mid WHEN
(cast(age#45 as int) > 51) THEN old ELSE kid END AS customer_group#57]
               +- *(1) Filter (isnotnull(city#49) AND (city#49 = boston))
                  +- *(1) ColumnarToRow
                     +- FileScan parquet [cust_id#43,name#44,age#45,gender#46,birthday#47,zip#48,city#49] Batched: true, DataFilters:
[isnotnull(city#49), (city#49 = boston)], Format: Parquet, Location: InMemoryFileIndex(1 paths)
[file:/Users/afaqueahmad/Documents/youtube/spark-experiments/data/data_..., PartitionFilters: [], PushedFilters: [IsNotNull(city),
EqualTo(city,boston)], ReadSchema: struct<cust_id:string,name:string,age:string,gender:string,birthday:string,zip:string,city:string>

+----------+--------------+---+------+---------+-----+------+--------------+-------------+-----------+
|cust_id   |name          |age|gender|birthday |zip  |city  |customer_group|test_column_2|birth_month|
+----------+--------------+---+------+---------+-----+------+--------------+-------------+-----------+
|C007YEYTX9|Aaron Abbott  |34 |Female|7/13/1991|97823|boston|mid           |test_column_2|13         |
|C08XAQUY73|Aaron Lambert |54 |Female|11/5/1966|75218|boston|old           |test_column_2|5          |
|C094P1VXF9|Aaron Lindsey |24 |Male  |9/21/1990|29399|boston|young         |test_column_2|21         |
|C097SHE1EF|Aaron Lopez   |22 |Female|4/18/2001|82129|boston|young         |test_column_2|18         |
|C0DTC6436T|Aaron Schwartz|52 |Female|7/9/1962 |57192|boston|old           |test_column_2|9          |
+----------+--------------+---+------+---------+-----+------+--------------+-------------+-----------+
only showing top 5 rows
```

✍️ Why df2 is now almost free

match the circles in the output above

- ① `InMemoryTableScan` ⟶ the **same** cached block df1 used. One copy, two readers.
- ② `CASE WHEN ... AS customer_group#57` ⟶ sits **inside** the InMemoryRelation — so it was computed once, when the cache was filled, not again for df2.
- ③ `split(birthday#47, /, -1)[1] AS birth_month#659` ⟶ df2's only new column.

Without the cache this plan reads the parquet, re-runs `Filter (city = boston)` and rebuilds `customer_group` a **second** time — the whole lineage, from scratch.

measured here	no cache	cached
Scan parquet ⟶ rows	5,000	0
size of files read	149.9 KiB	0
Filter ⟶ rows	517	0
InMemoryTableScan ⟶ rows	–	517
query duration	333 / 72 ms	48 / 38 ms

💡 Spark 4.3.0, local[12], customers.parquet, the two `count()` queries of each run.

`StorageLevel` Types:

(As of Spark `3.4`)

- `DISK_ONLY`: CPU efficient, memory efficient, slow to access, data is serialized when stored on disk
- `DISK_ONLY_2`: disk only, replicated 2x
- `DISK_ONLY_3`: disk only, replicated 3x
- `MEMORY_AND_DISK`: spills to disk if there's no space in memory
- `MEMORY_AND_DISK_2`: memory and disk, replicated 2x
- `MEMORY_AND_DISK_DESER` (default): same as `MEMORY_AND_DISK`, deserialized in both for fast access
- `MEMORY_ONLY`: CPU efficient, memory intensive
- `MEMORY_ONLY_2`: memory only, replicated 2x - for resilience, if one executor fails

Note:

- `SER` is CPU intensive, memory saving as data is compact while `DESER` is CPU efficient, memory intensive
- Size of data on disk is lesser as data is in serialized format, while deserialized in memory as JVM objects for faster access

When to use what?

Storage Level	Space used	CPU time	In memory	On-disk	Serialized
MEMORY_ONLY	High	Low	Y	N	N
MEMORY_ONLY_SER	Low	High	Y	N	Y
MEMORY_AND_DISK	High	Medium	Some	Some	Some
MEMORY_AND_DISK_SER	Low	High	Some	Some	Y
DISK_ONLY	Low	High	N	Y	Y

✍️ Why each row of that chart looks like that

level	the one sentence	pay with
MEMORY_ONLY	JVM objects, straight in RAM — nothing to decode on the way out	RAM. Blocks that don't fit are **silently dropped** and recomputed
MEMORY_ONLY_SER	same place, packed into byte arrays — about 2–5× smaller	CPU: every read has to deserialise
MEMORY_AND_DISK	RAM first, overflow spills to local disk instead of being thrown away	medium: RAM hits are fast, spilled blocks cost a disk read
MEMORY_AND_DISK_SER	the compact version of the same deal — more fits in RAM, so fewer spills	CPU on every read
DISK_ONLY	never in RAM; always serialised bytes on local disk	CPU + disk I/O on every read — but it never evicts

Read the chart as two dials, not five choices:

Two independent dials

SER ⟶ DESER — small + slow / big + fast

MEMORY ⟶ DISK — fast, can be evicted / slow, survives

× replication (1x / 2x / 3x) = same cost, N times

⚠ **Gotcha:** For **DataFrames** the chart is the RDD-era one. Spark SQL always stores a compressed columnar form, so `MEMORY_ONLY` on a DataFrame still shows up as *Memory Serialized* in the UI. Measured on the same 5,000 rows: **DESER 297,224 B** vs **SER 168,815 B**.

💡 Space and CPU trade against each other; memory and disk trade speed against survival. Replication (_2, _3) multiplies both costs and buys only resilience.

```
df_base.unpersist()
df_base.persist(StorageLevel.MEMORY_ONLY)

df2 = (
    df_base
    .withColumn("test_column_1", F.lit("test_column_1"))
    .withColumn("birth_year", F.split("birthday", "/").getItem(2))
)

df1.show(5, False)
```

✍️ Changing the storage level

- `unpersist()` then `persist(level)` — that order is required.
- `unpersist()` is **eager**: it drops the blocks right away. Everything downstream goes back to recomputing.
- After it, `df.storageLevel` is `StorageLevel.NONE`.

⚠ **Gotcha:** Calling `persist(ANOTHER_LEVEL)` on a DataFrame that is already cached **does not raise and does not change anything.** Tested here: a DISK_ONLY DataFrame asked for MEMORY_ONLY stayed Disk Serialized 1x Replicated. Silent no-op — the reason the cell above unpersists first.

⚠ **In this cell:** it rebuilds `df2` with df1's columns and then shows `df1`. The new df2 is never used — harmless in the video, but don't copy the pattern.

💡 Long notebook / long job? `unpersist()` as soon as the reuse is over, so the block manager can give the memory back.

```
spark.stop()
```

✍️ The whole chapter in five lines

- Spark is lazy: an action replays the **entire** lineage, every time.
- Two actions on one base = that base built twice.
- `cache()` cuts the lineage: the base becomes an `InMemoryRelation` and everything downstream reads it.
- `cache()` is `persist()`; persist is the one that takes an argument.
- Measured on a real 871 MB shuffle here: 3.19 s ⟶ 0.51 s on the reuse, **6.2× faster.**

💡 Interview line: "cache is a bet that recomputation costs more than memory. If the DataFrame is read once, you lose the bet."

Storage levels — the decision table

One shared example throughout: a DataFrame whose cached form is **10 GB deserialized / 3 GB serialized**, on a cluster with **6 GB** of storage memory per executor.

Level	Rule of thumb	On the 10 GB / 6 GB example	What ends up where
`MEMORY_ONLY` RDD default	Fastest reads, biggest footprint. Only when the cache **comfortably** fits.	6 GB of blocks are kept, 4 GB are **dropped**. Those partitions are recomputed from the lineage on every read — no warning, just a slow query.	memory 6 GB | 4 GB lost recomputed on every read
`MEMORY_AND_DISK_DESER` cache() on a DataFrame	The sane default. Fast where it fits, spilled where it doesn't, never recomputed.	6 GB stay in memory, 4 GB go to the executor's local disk. Nothing is lost.	memory 6 GB | disk 4 GB nothing recomputed
`MEMORY_AND_DISK_SER`	The one to switch to when the cache is too big. Trades CPU for a far better hit rate.	The whole 3 GB serialized form fits in 6 GB — **zero** disk, and 3 GB of memory left for execution. Every read pays a deserialise.	cache 3 GB | 3 GB free + CPU on every read
`DISK_ONLY`	When memory is needed for execution, or the source is non-deterministic / very expensive to re-read.	3 GB on local disk, 6 GB of memory free for shuffles and joins. Slowest reads, but the block is guaranteed present.	memory free | disk 3 GB slowest reads, never evicted
`*_2` / `*_3`	Resilience only. Doubles or triples the cost; buys you not re-running the lineage when an executor dies.	20 GB instead of 10 GB for `MEMORY_ONLY_2`. Worth it only for long jobs on spot / preemptible instances.	exec A 10 GB | exec B 10 GB same data twice = survives one loss

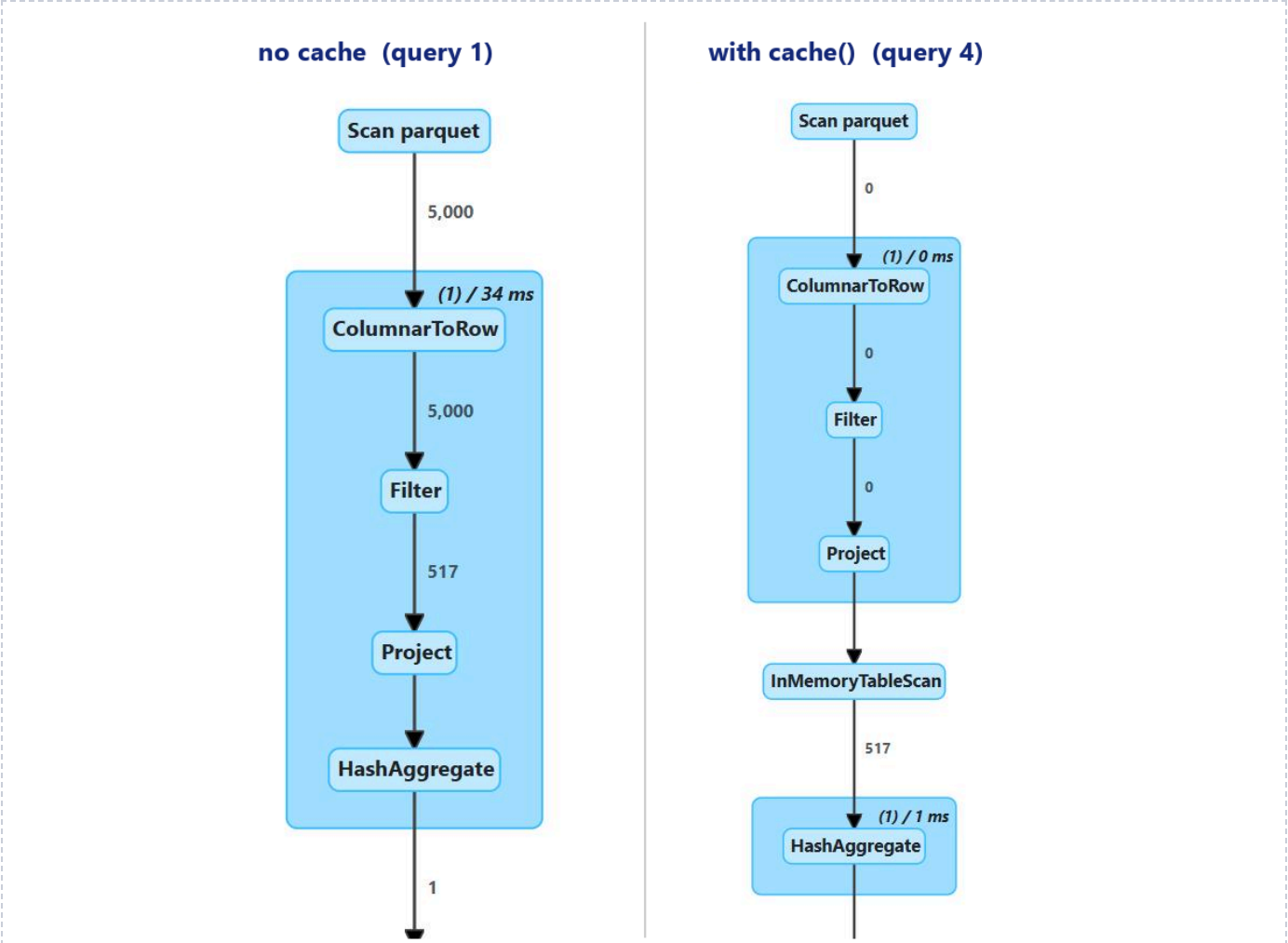
💡 Order to try things in: **`cache()`** → check the Storage tab → if it is not 100% cached, `MEMORY_AND_DISK_SER` → if memory is still the bottleneck, `DISK_ONLY` or stop caching and let the optimizer re-read parquet.

Caching — extra notes

Measured on this machine: Spark 4.3.0, Java 17, `local[12]`, driver 6 GB, AQE off, `spark/4_caching.ipynb` replayed by `scripts/caching_experiment.py`. Raw output in `notes/caching_run/`.

1 · Before and after, in the SQL tab

Same two `count()` actions on `df1` and `df2`. The only difference between the runs is one call to `df_base.cache()`.



Spark UI metric	no cache (query 1 / 2)	cached (query 4 / 5)
`Scan parquet` → number of output rows	5,000	**0**
`Scan parquet` → number of files read	1	**0**
`Scan parquet` → size of files read	149.9 KiB	**0**
`Filter` → number of output rows	517	0
`InMemoryTableScan` → number of output rows	—	**517**
query duration	333 ms, 72 ms	48 ms, 38 ms

The `Scan parquet` node is **still in the cached plan** — it is the recipe hanging under `InMemoryRelation`. It reports zero because it was not executed.

Physical plan, side by side (trimmed)

```
no cache — the file is read again
*(1) Project [... customer_group#8,
       test_column_1, birth_year#10]
```

```
cached — the file is only the recipe
*(1) Project [... customer_group#74,
       test_column_1, birth_year#328]
```

```
+- *(1) Filter (city#6 = boston)
   +- *(1) ColumnarToRow
      +- FileScan parquet [...]
         PushedFilters: [EqualTo(city,boston)]
```

```
+- InMemoryTableScan [...]
   +- InMemoryRelation [...],
      StorageLevel(disk, memory,
                   deserialized, 1 replicas)
      +- *(1) Project [CASE WHEN ...]
         +- *(1) Filter (city#72 = boston)
            +- FileScan parquet [...]
```

And when the base is genuinely expensive

The customers file is 150 KiB — too small to feel. Repeated on the 871 MB `transactions.parquet`, grouped by `(cust_id, expense_type)` (one real shuffle) and consumed twice:

	no cache	cached
fill the cache (first action)	—	1.70 s
two consumers of the base	3.19 s	**0.51 s**
total, this run	3.19 s	2.21 s
cost of each *further* reuse	~1.6 s	**~0.25 s**

Cached base: 869,976 bytes across 32 partitions. The speed-up on the reuse is **6.2×**; the speed-up on the *whole job* is only 1.4×, because the first pass still has to happen. That gap is the whole economics of caching.

> **One-liner:** caching does not make the first pass faster. It makes every pass after the first one nearly free.

2 · `cache()` vs `persist()`

There is only one mechanism. `cache()` is `persist()` with the default storage level.

In the Spark source, `def cache() = persist()`. Nothing else differs: same lazy marking, same `InMemoryRelation`, same block manager, same `unpersist()`.

Measured here

```
df.cache()                       -> Disk Memory Deserialized 1x Replicated
df.persist()                     -> Disk Memory Deserialized 1x Replicated
df.persist(MEMORY_AND_DISK_DESER) -> Disk Memory Deserialized 1x Replicated
df.persist(DISK_ONLY)            -> Disk Serialized 1x Replicated
cache() == persist() == persist(MEMORY_AND_DISK_DESER)?  True
```

The default is not the same for RDDs and DataFrames

API	`cache()` / `persist()` default	why
RDD	`MEMORY_ONLY`	an RDD is opaque JVM objects; Spark cannot compress what it cannot describe, and dropping a partition is cheap.
DataFrame / Dataset / SQL `CACHE TABLE`	`MEMORY_AND_DISK_DESER`	Spark SQL knows the schema, so it stores a compressed columnar block and can spill it to disk rather than throw away work the optimizer already paid for.

> **Interview trap.** "What is the default storage level of `cache()`?" is only answerable once you ask *on what*. Saying `MEMORY_ONLY` for a DataFrame is the single most common wrong answer.

So when do you use which

use	when
`cache()`	the default is fine — which is most of the time. Shorter to read, and it is what the next person expects.
`persist(level)`	you have a reason to deviate: memory is tight (`MEMORY_AND_DISK_SER`), the block must never be recomputed because it came from a non-deterministic source (`DISK_ONLY`), or executors die often enough that re-running the lineage hurts (`_2`).

> *"They are the same call — `cache()` is `persist()` with no argument. I reach for `persist` only when I want a non-default storage level: serialized when the cache is too big for memory, disk-only when recomputing is expensive or non-deterministic, replicated when I am losing executors. The one thing worth remembering is that the default differs by API: `MEMORY_ONLY` for RDDs, `MEMORY_AND_DISK_DESER` for DataFrames."*

> **One-liner:** `cache()` is the convenience name; `persist()` is the steering wheel.

3 · The storage-level chart, row by row

```
Storage Level        Space used  CPU time  In memory  On-disk  Serialized
MEMORY_ONLY              High        Low       Y          N        N
MEMORY_ONLY_SER          Low         High      Y          N        Y
MEMORY_AND_DISK          High        Medium    Some       Some     Some
MEMORY_AND_DISK_SER      Low         High      Some       Some     Y
DISK_ONLY                Low         High      N          Y        Y
```

Every cell in that table falls out of two facts:

1. **Serialized bytes are small and slow; deserialized JVM objects are big and fast.** A JVM object carries a header, pointers and padding — typically 2–5× the size of the same data packed into a byte array. Serialized form has to be decoded on every read, which is CPU.
2. **Memory is fast but finite; disk is slow but kept.** A block that will not fit in memory is either dropped (and recomputed later) or spilled.

row	why Space	why CPU	why the Y/N columns
`MEMORY_ONLY`	**High** — JVM objects, no packing	**Low** — nothing to decode, read straight out	memory Y / disk N / ser N. Blocks that don't fit are dropped and *recomputed on demand* — no error, just a slow query later.
`MEMORY_ONLY_SER`	**Low** — packed byte arrays	**High** — deserialise on every single read	same memory-only eviction behaviour, just more of your data fits.
`MEMORY_AND_DISK`	**High** — the in-memory part is still deserialized objects	**Medium** — RAM hits cost nothing, spilled blocks cost a disk read *and* a deserialise	"Some" everywhere because it is a mix: Spark keeps what fits in memory deserialized and writes the overflow to disk serialized.
`MEMORY_AND_DISK_SER`	**Low**	**High**	same split, but compact, so far *fewer* blocks ever reach the disk. Usually the best choice when a big cache does not fit.
`DISK_ONLY`	**Low** — on disk, always serialized	**High** — disk I/O plus deserialise, every read	memory N / disk Y / ser Y. Nothing can be evicted, so the block is guaranteed to be there. Pay for that guarantee in latency.

The same thing, measured (customers.parquet, 5,000 rows)

level requested	in memory	on disk	how the UI labels it
`MEMORY_AND_DISK_DESER` (`cache()`)	297,224 B	0	Disk Memory Deserialized 1x Replicated
`MEMORY_ONLY`	168,815 B	0	Memory **Serialized** 1x Replicated
`MEMORY_ONLY_2`	168,815 B	0	Memory Serialized **2x** Replicated
`MEMORY_AND_DISK`	168,815 B	0	Disk Memory Serialized 1x Replicated
`DISK_ONLY`	0	168,815 B	Disk Serialized 1x Replicated
`DISK_ONLY_3`	0	168,815 B	Disk Serialized **3x** Replicated

Two things to read off that table. **297 KB vs 169 KB** is the deserialized/serialized tax, exactly the "Space used High vs Low" column. And `MEMORY_AND_DISK` shows **0 bytes on disk** — not a bug: memory was big enough, so nothing spilled. Disk in the name is a fallback, not a destination.

> **Two caveats on the chart.** (a) It is the **RDD** table. Spark SQL always caches a compressed columnar block, so `MEMORY_ONLY` on a DataFrame still reports "Serialized"; the only deserialized DataFrame level is `MEMORY_AND_DISK_DESER`. (b) `_2` / `_3` replication did nothing measurable above because `local[*]` has a single executor — you only get real replicas on a real cluster.

> **One-liner:** start at `cache()`; move to `MEMORY_AND_DISK_SER` when it does not fit; move to `DISK_ONLY` only when recomputing is worse than reading from disk.

4 · Lazy evaluation — why the base is built twice

Transformations build a DAG and return immediately. Nothing runs until an action (`show` , `count` , `collect` , `write`) asks for rows. At that moment Spark walks the **whole** lineage from the source and executes it.

A DataFrame is a *recipe*, not a *result*. `df_base` is not “a table of 517 Boston customers sitting somewhere”; it is the sentence “read this parquet, keep city = boston, add customer_group”. Assigning it to a variable and using it twice reuses the sentence, not the table — so the sentence is executed twice.

```
df_base = read → filter(city = boston) → CASE WHEN customer_group

df1 = df_base → test_column_1 → birth_year  → .show()   # replays all of df_base
df2 = df_base → test_column_2 → birth_month → .show()   # replays all of df_base again
```

`cache()` is how you turn the recipe into a result. After the first action the `InMemoryRelation` holds real rows, and every DataFrame built on `df_base` starts from those rows.

The job that appears before you call any action

From the video, and worth knowing: `spark.read.parquet(...)` shows up as a job in the UI even though reading is not an action. That job is schema inference — Spark opens the footer to learn the columns and types. Its **input size is 0**, which is how you recognise it. (With a schema supplied by hand, or for a table in a metastore, it does not appear at all.)

> **One-liner:** an action replays the entire lineage; a cache is the only thing that shortens it.

5 · When *not* to cache, and what bites people

situation	what to do
The DataFrame is read once	Don't cache. You pay the write and the memory for nothing.
The base is a plain scan with a pushed-down filter	Usually don't. Re-reading columnar parquet with a pushed predicate can be cheaper than materialising a cache.
The base is wide and you only use three columns downstream	`select()` first, *then* cache. Caching before projecting stores columns nobody reads.
The expensive part is a shuffle you do once	Spark already keeps shuffle files; a cache on top of a single shuffle often adds little.
The reuse spans several jobs or a loop (ML, iterative joins)	Cache — this is the case the feature exists for.

Gotchas, in the order they catch people

1. `cache()` is lazy. `df.cache()` alone does nothing; the next action fills it. `df.count()` is the usual way to force it.
2. `show(5)` materialises only the partitions it touched, so the cache can be *partially* filled and the Storage tab shows “50% cached”.
3. `persist(other_level)` on an already-cached DataFrame is a **silent no-op** — no error, no change. Call `unpersist()` first.
4. Caching a DataFrame does not cache the ones you built from it; each has its own identity.
5. `MEMORY_ONLY` blocks that do not fit are dropped silently and recomputed. The symptom is a query that is mysteriously slow, not an error.
6. Cache competes with execution memory for the same unified pool. An over-eager cache can push shuffles and joins into spilling.
7. Never cached forever: `unpersist()` once the reuse is over, especially in notebooks and long-running apps.

> **One-liner:** cache is a bet that recomputation costs more than memory — check the bet in the Storage tab, not in your head.

6 · Interview answers

“What does `cache()` do, and when would you use it?”

> *“Spark is lazy, so every action replays the whole lineage. If one DataFrame feeds several downstream DataFrames, that lineage runs once per action. `cache()` marks the DataFrame so that after the first action its rows are stored in the block manager as an `InMemoryRelation`; everything downstream then reads `InMemoryTableScan` instead of re-scanning the source. I use it when a non-trivial intermediate — typically something that cost a shuffle — is read two or more times.”*

“How do you *prove* the cache is working?”

> *“Three places. The physical plan gains an `InMemoryTableScan` over an `InMemoryRelation`. In the SQL tab the `Scan parquet` node drops to 0 rows and 0 bytes — it is still drawn, because it is the recipe for rebuilding, but it is not executed. And the Storage tab shows the RDD, its storage level, how many partitions are cached and the bytes in memory and on disk.”*

“Difference between `cache` and `persist`?”

> *“None, except the argument. `cache()` calls `persist()` with the default level, which is `MEMORY_ONLY` for RDDs and `MEMORY_AND_DISK_DESER` for DataFrames. `persist` lets me pick: serialized to fit more in memory at a CPU cost, disk-only when the block must never be recomputed, `_2` for a replica when executor loss is a real risk.”*

“Your cached DataFrame does not fit in memory. What happens?”

> *“It depends on the level. With `MEMORY_ONLY` the blocks that do not fit are simply not stored, and are recomputed from the lineage next time they are needed — silently. With a `MEMORY_AND_DISK` level the overflow spills to the executor's local disk instead. That is why `MEMORY_AND_DISK_SER` is the usual answer for a cache that is too big: the compact form means less of it ever reaches disk.”*

Quick-fire

question	answer
Is `cache()` an action?	No. It is lazy. The next action fills it.
Is `unpersist()` lazy?	No, it is eager — blocks go immediately.
Default level of `spark.sql("CACHE TABLE t")`?	`MEMORY_AND_DISK_DESER`, and that statement is **eager** unless you write `CACHE LAZY TABLE`.
Does caching survive a `spark.stop()`?	No. It lives in the executors' memory and local disk for the life of the application.
Cache vs checkpoint?	Cache keeps the lineage (it can rebuild). Checkpoint writes to reliable storage and *truncates* the lineage — use it to break very long DAGs, not for speed.
Cache vs broadcast?	Cache reuses a DataFrame across actions. Broadcast ships a small table to every executor for a join. Different problems.

> **Study next in this repo:** `5_0_partitioning.ipynb`, `6_0_bucketing.ipynb`, then `1_data_skew/` — caching interacts with all three, because the thing worth caching is usually the output of a shuffle.